
Big Data

Management and Analytics

TU Clausthal, Institut für Informatik

Summer Semester 2026

Due Date: 17 June 2026, 16:00

Instructions — Please Read Carefully

In this task you will build a data pipeline that fetches data from the **Star Wars API (SWAPI)** and loads it into a **Neo4j** graph database using one of the official drivers.

Further Reading:

- ▷ [SWAPI — Star Wars API](#)
- ▷ [Neo4j Drivers — pick any officially supported language](#)

Assignment Rules

- The assignment has a total of **10 marks**
- Work in your assigned teams of **up to 2 members**
- Submit your solutions **via Moodle**
- Solutions submitted after the deadline will automatically receive **0 marks**
- Copying solutions from or sharing with other students will lead to **exclusion from the course**
- The use of language software is **not permitted**; each group member must understand every part of the solution
- Document your solutions with **screenshots** and attach a **TEXT file** with your code/queries
- If anything is unclear, state your assumptions explicitly to make your solution understandable

Task 1 **5 Marks**

Note: The API exposes six resource endpoints under the base URL `https://swapi.info/api/` `people`, `films`, `planets`, `species`, `starships`, and `vehicles`. Each endpoint returns the full list of objects for that resource.

(a) **Examine and explain the API response.** Query the resource endpoints and inspect at least one sample object from each. In your report:

- show a representative JSON response and list the fields each resource returns;
- distinguish the *scalar* fields (plain values such as `name` or `height`) from the *reference* fields that hold URLs pointing to other resources (e.g. `homeworld`, `films`, `pilots`);
- explain how relationships between entities are represented in the raw JSON, and how you would follow a reference from one resource to another.

(b) **Design the graph schema.** Based on your examination, design a knowledge graph model for Neo4j and **draw a schema diagram** showing the node labels and the directed, labelled relationships between them. Justify your design choices:

- which resources become *nodes*, and which fields become node *properties*;
- which reference fields become *relationships*, and why you modelled them as edges rather than as properties;
- your choice of relationship *direction* and *naming*;
- any fields you decided to drop, rename, or transform, and the reasoning behind it.

(c) **Implement the loader.** Write program that realises the schema you designed. It should:

- fetch all six resource types from the API;
- connect to Neo4j using the official `neo4j` driver;
- create the nodes with their properties, then create the relationships;
- make the import *idempotent* (e.g. uniqueness constraints together with `MERGE`), and explain how you derive a stable identifier for each entity.

Deliverables: Submit a `.txt` file containing your code, the schema diagram, and screenshots showing the populated graph in Neo4j.

Task 2 5 Marks

- (a) **Three tallest characters per film.** For each film, return its **three tallest characters** (character name and height). Ignore characters whose height is "unknown".
- Output columns: `film`, `character`, `height`.
- (b) **Skin colors across species.** Across all species, list each **distinct skin color** and the number of species that have it.
- Output columns: `skin_color`, `species_count`.
- (c) **Average film appearances per resident, by planet.** Every character has a home planet and appears in some films. We want, for each planet, the **average over its residents of how many films each resident appears in** (e.g. a planet with two residents — one in 3 films, one in 1 — has value $(3 + 1)/2 = 2.0$).
- Base query.** Return `planet`, `averageFilms` (rounded to **3 decimals**), and `residentsCounted` (the number of residents the average is taken over). Order by `averageFilms` descending, then `planet` ascending.
 - Your assigned row.** Let $N = (\text{last two digits of your student ID}) \bmod 10$, then $+1$ (a number from 1 to 10). From your result, report the planet at **position** N , with its exact `averageFilms` and `residentsCounted`.
 - The grouping trap.** A classmate writes the same query but groups with `WITH planet, count(f)` instead of `WITH planet, person, count(f)`. For *your assigned planet* from (ii), give the value their version produces alongside the correct one, and explain in 2–3 sentences what their query actually computes and why it is wrong.

Hint: think carefully about what each *WITH* groups by — the grouping key determines the grain of every aggregation.

Deliverables: Submit a `.txt` file containing all Cypher queries and your driver program(s), together with screenshots showing the execution results in Neo4j.